

ZYRAA Backend Engineering Handbook

Comprehensive backend code walkthrough and architecture reference

Generated: 2026-05-09 22:26:57

Files covered: 28

Total source lines covered: 4408

Classes and functions indexed: 274

Scope: backend/app

This handbook explains the FastAPI startup path, SQLAlchemy behavior, routers, CRUD helpers, schemas, models, service layer, prompt automation, assignment intelligence, and the practical meaning of the SQL logs produced during runtime.

System Overview

The backend is a layered FastAPI application. The core flow is: HTTP request -> dependency resolution -> validation -> ORM/service logic -> commit/refresh -> JSON response.

- Entry and app wiring live in `main.py`, `config.py`, and `database.py`.
- Authentication lives in `auth.py` plus `api/v1/dependencies.py`.
- Pydantic schemas define request and response contracts.
- SQLAlchemy models define the persistent domain graph.
- CRUD helpers centralize routine query patterns.
- Routers translate HTTP requests into business operations.
- Services implement assignment, sprint, PR health, prompt planning, local LLM access, and email delivery.

Startup, Sessions, and SQL Logs

1. Runtime Startup Sequence

- Uvicorn imports `app.main:app`. Import-time code builds the FastAPI instance, registers CORS, and mounts the API router tree.
- When FastAPI fires the startup event, the app calls `create_database_if_not_exists()` against SQL Server `master`. This checks `sys.databases` and creates `JiraDB` if needed.
- The startup hook then imports `app.models`, causing SQLAlchemy model classes to register themselves inside `Base.metadata`.
- Next, `Base.metadata.create_all(bind=engine)` asks SQLAlchemy to inspect existing tables and create any missing ones. This emits metadata queries and possibly DDL.
- Finally, `seed_reference_data()` inserts required rows such as `Epic`, `Story`, `Task`, `Bug`, `To Do`, `Done`, and priority rows. Without these rows, issue creation endpoints would fail validation or default lookup logic.

2. Why SQL Logs Appear So Frequently

- The engine in `database.py` is created with `echo=True`. That setting logs SQL text, parameters, and transaction markers for every database interaction.
- SQLAlchemy starts implicit transactions for many operations. Even a read-only request may show `BEGIN (implicit)` followed by one or more `SELECT` statements.
- If the request finishes without an explicit commit, SQLAlchemy or the DBAPI connection may emit `ROLLBACK` when the session is cleaned up. In this app that is normal for read-only flows; it does not mean data loss occurred.
- Many endpoints use `joinedload(...)` to prefetch relationships. That changes SQL shape and often produces wider queries to reduce later per-row fetches.
- Pagination on SQL Server requires ordering. Earlier in development, missing `ORDER BY` clauses caused MSSQL compilation errors when SQLAlchemy generated `OFFSET/LIMIT` queries.

3. Request Lifecycle in This App

- The browser sends a request with a bearer token stored in local storage by the frontend.
- FastAPI matches the route and resolves dependencies. Protected endpoints call `get_db()` to create a SQLAlchemy session and `get_current_user()` to decode the JWT and load the user row.
- The router performs validation through Pydantic schemas, then either uses raw SQLAlchemy ORM calls or delegates to the CRUD/service layers.
- The endpoint usually commits changes, refreshes ORM objects to hydrate DB-generated values, and returns either ORM models or custom dictionaries for JSON serialization.
- When the request ends, the DB session is closed. If nothing was committed in a read flow, SQLAlchemy may roll back the open transaction during cleanup.

4. How the Agent System Is Actually Implemented

- There are two distinct automation families: the workflow agents in ``agent_service.py`` and the prompt planner/creator path driven by ``langchain_service.py`` plus ``run_prompt_automation()``.
- The workflow side is deterministic first. Assignment, sprint rotation, and PR analysis all work even with no model installed.
- Ollama is used as an optional narrative layer for executive summaries. LangChain is used as an optional structured JSON planner for prompt automation.
- LangGraph is only capability-checked today. The system reports whether it is installed, but the current workflow execution is still hand-orchestrated Python, not a built ``StateGraph``.
- Human approval is modeled explicitly with ``AgentAction`` rows, which allows the app to stage proposed actions and execute them later only after approval.

Authentication and Request Flow

- Passwords are hashed with Passlib before persistence.
- JWT tokens store the username in the sub claim.
- Protected routes resolve the token to a real database user through `get_current_user()`.
- Each request gets its own SQLAlchemy session from `get_db()`.
- Read-only requests can still show BEGIN and ROLLBACK in logs because the session opens and then cleans up an implicit transaction.

Data Model Walkthrough

Identity and access

- ``User`` stores accounts and is the anchor for reported issues, assigned issues, comments, worklogs, notifications, and project roles.
- ``ProjectRole`` and ``Permission`` lay groundwork for authorization, although current endpoint checks are still relatively simple.

Issue tracking core

- ``Project`` groups issues and boards under a project key.
- ``IssueType``, ``IssuePriority``, ``IssueStatus``, and ``Resolution`` are lookup tables used by issue rows.
- ``Issue`` is the central transactional record and connects almost every other subsystem together.
- ``Label``, ``Component``, and ``Version`` provide issue classification and release metadata.

Collaboration and workflow

- ``IssueComment``, ``Worklog``, ``IssueAttachment``, ``IssueHistory``, and ``Favorite`` represent user interaction around work items.
- ``Board``, ``BoardColumn``, and ``Sprint`` support kanban/scrum workflow organization.
- ``IssueLink`` generalizes relationships such as parent-child links used for Epic membership.

Automation and analytics

- ``UserSkill``, ``UserAssignmentHistory``, and ``IssueRequirement`` feed assignment scoring.
- ``AgentMemory`` and ``AgentAction`` persist automation context and approval-gated tasks.
- ``PullRequest``, ``CodeReview``, and ``BugReport`` connect source control health checks back into issue tracking.

Agents and Automation

- The app mixes deterministic automation with optional AI support from local Ollama models.

- Workflow automation in `agent_service.py` handles assignment recommendations, sprint maintenance, PR health analysis, executive summaries, pending approval actions, and prompt-driven object creation.
- LangChain is optional and is used only for structured prompt planning when available.
- LangGraph is only capability-checked today; the actual workflow orchestration is still imperative Python over an AgentState dictionary.
- Prompt automation can create projects, boards, multiple issues, parent-child links, assignees, notifications, and assignment emails from one sentence.

End-to-End Request Examples

POST `/api/v1/auth/login`

- Reads credentials from JSON or form data.
- Fetches the user by username.
- Verifies the password hash with Passlib.
- Rejects inactive users.
- Creates a JWT whose ``sub`` claim stores the username.
- Returns ``{access_token, token_type}`` to the frontend.

POST `/api/v1/issues/`

- Validates the request body with ``IssueCreate``.
- Resolves the project by key and the issue type by enum value.
- Loads the default ``To Do`` status row.
- Optionally resolves assignee, component, version, and priority.
- Creates the issue row and then attaches labels using ``crud.label.get_or_create(...)``.
- If ``auto_assign`` is true and there is no manual assignee, runs the assignment recommender and updates the issue assignee.
- Serializes a custom issue response dictionary, optionally including recommendation metadata.

POST `/api/v1/agents/prompt/execute`

- Accepts a natural-language prompt.
- Builds an automation plan via LangChain+Ollama when available, otherwise via regex fallback parsing.
- Optionally creates a project, a board, one or more issues, issue-chain links, assignees, notifications, and emails.
- Returns both the interpreted plan and the concrete objects that were created.

File-by-File Reference

Each entry below includes the module purpose, key implementation notes, and the top-level classes/functions parsed from the source.

backend/app/__init__.py

Line count: 3

Purpose: Package marker for the `app` Python package.

- This file is intentionally minimal. Its main role is making `app` importable as a package namespace.

No parseable classes or functions were extracted for this file.

backend/app/api/__init__.py

Line count: 0

Purpose: This module exists in the backend tree but does not yet have a custom narrative note in the generator.

No parseable classes or functions were extracted for this file.

backend/app/api/v1/__init__.py

Line count: 11

Purpose: Versioned API composition point.

- Creates the `/api/v1` router and includes the auth, users, projects, issues, boards, and agents subrouters.
- This keeps route registration centralized so the root app only mounts one router.

No parseable classes or functions were extracted for this file.

backend/app/api/v1/agents.py

Line count: 124

Purpose: Public interface for the automation subsystem.

- The status endpoint reports whether local AI, LangChain, LangGraph, and SMTP are available, which is useful for debugging hybrid deterministic/AI behavior.
- The automation endpoint invokes the internal state-driven workflow for assignment, sprint maintenance, PR health checks, or a full scan.
- The prompt execution endpoint is the natural-language project/board/issue creation interface.
- Approve/reject endpoints operate on persisted `AgentAction` rows, which lets the system stage agent recommendations before applying them.

Kind	Name	Line
class	RunAutomationAgentRequest	24
class	AskAgentRequest	30
class	PromptAutomationRequest	34
function	agent_status	39
async function	run_automation	54
async function	ask_agent	69
async function	execute_prompt	78
function	approve_action	87
function	reject_action	109

backend/app/api/v1/auth.py

Line count: 82

Purpose: Login and registration endpoints.

- The login route accepts both form-encoded and JSON credentials, which makes it compatible with browser form posts and Axios JSON calls.
- On successful login it returns a bearer token only; the frontend stores and reuses that token for later requests.
- Registration performs uniqueness checks on username and email, hashes the password, and writes the `User` row directly.

Kind	Name	Line
async function	login	15
function	register	55

backend/app/api/v1/boards.py

Line count: 159

Purpose: Board and board-column management plus board issue grouping.

- Boards are linked to projects by project key at creation time, then stored with the project's numeric foreign key.
- Columns can map to statuses, which is how the board turns workflow states into visual lanes.
- The board issue endpoint loops through columns and queries issues by each mapped status. That is straightforward but can become chatty at scale because it issues one query per mapped column.

Kind	Name	Line
function	create_board	15
function	read_project_boards	37
function	read_board	51
function	update_board	59
function	create_board_column	72
function	read_board_columns	103
function	read_board_issues	121

backend/app/api/v1/dependencies.py

Line count: 42

Purpose: Reusable authentication dependencies for protected endpoints.

- Defines `oauth2\_scheme` with `/api/v1/auth/login` as the token URL so FastAPI's OpenAPI docs understand how bearer auth is acquired.
- `get\_current\_user()` decodes the token, reads the `sub` claim as a username, and looks the user up in the database.
- `get\_current\_active\_user()` is a second guard that rejects inactive accounts.

Kind	Name	Line
function	get_current_user	12
function	get_current_active_user	38

backend/app/api/v1/issues.py

Line count: 590

Purpose: Largest router in the application. Handles issue create/read/update/delete, comments, worklogs, links, PR linking, auto-assignment, serialization, and Epic parenting.

- The file contains helper functions such as ``generate_issue_key()``, ``serialize_issue()``, and Epic-link helpers. These exist because the app returns custom response dictionaries rather than raw ORM objects.
- Issue creation resolves project, issue type, default status, optional assignee, optional component/version, optional priority, and labels before committing the row.
- When ``auto_assign`` is enabled, the router delegates to ``recommend_issue_assignee()`` and optionally rewrites the assignee after the issue is initially created.
- Issue listing uses ``joinedload(...)`` for related objects. That reduces N+1 query explosions when serializing project, priority, status, assignee, reporter, component, version, and labels.
- Update logic is explicit rather than generic because enum values, assignee usernames, project moves, component scoping, version scoping, and label replacement all need custom handling.
- The Epic endpoint treats Epic membership as a special case of ``IssueLink`` with ``link_type='parent-child'``, but it wraps that rule in validation so the UI can treat it as a first-class feature.
- The PR endpoint creates a ``PullRequest``, runs code analysis, updates PR health, and creates bug reports from high-severity findings.

Kind	Name	Line
function	get_parent_epic	29
function	generate_issue_key	38
function	serialize_issue	52
function	validate_epic_assignment	86
function	replace_parent_epic_link	100
function	serialize_comment	116
function	serialize_worklog	129
function	create_issue	146
function	read_issues	234
function	read_issue	288
function	read_issue_by_key	296
function	update_issue	304
function	delete_issue	398
function	create_comment	408
function	read_issue_comments	426
function	create_worklog	446
function	read_issue_worklogs	471
function	create_issue_link	491
function	update_issue_epic	513
function	link_pull_request	558

backend/app/api/v1/projects.py

Line count: 110

Purpose: Project CRUD and project-scoped issue/stat endpoints.

- Creating a project stamps the current user as the project lead.
- Project issues are fetched through ``crud.issue.get_by_project(...)``, so paging and filters are applied at the CRUD layer.
- The stats endpoint deliberately calculates totals in Python after loading issues. That is easy to understand, but it is less efficient than pushing aggregates into SQL.

Kind	Name	Line
function	create_project	15
function	read_projects	37
function	read_project	47
function	update_project	55
function	read_project_issues	68
function	read_project_stats	96

backend/app/api/v1/users.psy

Line count: 65

Purpose: Legacy or stray duplicate of the users router kept in the tree with a non-standard extension.

- It is not imported by the live router composition file, so it does not participate in the running application.
- Its presence is worth documenting because it can confuse readers who search for user endpoints and find two copies.

No parseable classes or functions were extracted for this file.

backend/app/api/v1/users.py

Line count: 90

Purpose: User CRUD endpoints plus assignee search.

- The `/search` route is important for issue assignment UX. It performs an active-user filter plus `ILIKE` search on username, display name, and email.
- Update authorization is intentionally simple: a user can only update their own record.
- The create route still uses the CRUD layer, but it constructs a `UserCreate` object in a slightly awkward way because password hashing is handled outside the generic CRUD helper.

Kind	Name	Line
function	create_user	16
function	read_users	34
function	search_users	43
function	read_user	66
function	update_user	75

backend/app/auth.py

Line count: 41

Purpose: Authentication primitives: password hashing/verification and JWT creation/decoding.

- Passlib's `CryptContext` is configured with multiple schemes so older hashes remain readable if formats evolve.
- `verify\_password()` compares plain text credentials with the stored hash from the database.
- `create\_access\_token()` puts an `exp` claim into the token and signs it with the configured secret key and algorithm.
- `decode\_token()` returns the payload when the token is valid and `None` when validation fails; the dependency layer translates `None` into a 401.

Kind	Name	Line
function	verify_password	13
function	get_password_hash	18
function	create_access_token	23

Kind	Name	Line
function	decode_token	35

backend/app/config.py

Line count: 34

Purpose: Central settings object built with `pydantic\_settings.BaseSettings`.

- JWT, CORS, Ollama, and SMTP settings are all loaded through one typed configuration class.
- The `.env` file is the default source for overrides, but class defaults make local development work even without extra environment variables.
- The `database\_server` field exists, but the live database module currently builds its own hard-coded SQL Server URL instead of consuming this setting directly.

Kind	Name	Line
class	Settings	5
class	Config	30

backend/app/crud/__init__.py

Line count: 296

Purpose: Specialized CRUD objects for users, projects, issues, comments, worklogs, boards, sprints, labels, notifications, favorites, and board columns.

- This file is the persistence convenience layer. It keeps routine query patterns out of routers so controllers stay readable.
- The issue CRUD object adds project scoping, board issue lookup, and free-text search.
- Notification helpers centralize unread filtering and bulk mark-as-read behavior.
- The module exposes both named instances like `crud\_user` and namespaced aliases like `crud.user`, which is why imports can use either style.

Kind	Name	Line
class	CRUDUser	9
function	get_by_username	10
function	get_by_email	13
function	is_active	16
function	get_user_projects	19
class	CRUDProject	25
function	get_by_key	26
function	get_multi_by_lead	29
class	CRUDIssue	42
function	get_by_key	43
function	get_by_project	46
function	get_board_issues	68
function	search	98
class	CRUDComment	119
function	get_by_issue	120
class	CRUDWorklog	133
function	get_by_issue	134

Kind	Name	Line
function	get_by_user	146
class	CRUDBoard	159
function	get_by_project	160
class	CRUDSprint	173
function	get_by_board	174
function	get_active_sprints	186
class	CRUDLabel	197
function	get_by_name	198
function	get_or_create	201
class	CRUDNotification	208
function	get_by_user	209
function	mark_as_read	217
function	mark_all_as_read	226
class	CRUDFavorite	236
function	get_user_favorites	237
function	is_favorite	249
class	CRUDBoardColumn	258
function	get_by_board	259

backend/app/crud/base.py

Line count: 80

Purpose: Generic CRUD helper used by specialized CRUD classes.

- Implements `get`, `get\_multi`, `create`, `update`, and `delete` in a model-agnostic way.
- The helper assumes Pydantic-style objects expose `model\_dump(...)`. That design caused one of the earlier bugs when a SQLAlchemy model was accidentally passed into `create()`.
- `get\_multi` automatically adds an `ORDER BY` on the primary key when no explicit order is provided. That matters for SQL Server because OFFSET/LIMIT pagination requires ordering.

Kind	Name	Line
class	CRUDBase	10
function	__init__	11
function	get	14
function	get_multi	17
function	create	46
function	update	54
function	delete	75

backend/app/database.py

Line count: 168

Purpose: Owns the SQL Server connection string, SQLAlchemy engine, session factory, declarative base, database creation routine, and reference-data seeding.

- The module uses ``mssql+pyodbc`` against ``localhost\SQLEXPRESS02`` with ODBC Driver 18 and Windows trusted authentication.
- Two URLs exist: one for the application database (``JiraDB``) and one for ``master``. The ``master`` connection is used only for existence checks and initial ``CREATE DATABASE``.
- The ``SessionLocal`` factory creates request-scoped sessions. Each API dependency calls ``SessionLocal()``, yields it, and closes it afterward.
- The ``seed_reference_data()`` function ensures required lookup rows exist for issue types, priorities, statuses, and resolutions. Many higher-level endpoints assume these rows are present.
- A large share of the SQL logs the user sees come from this module because ``echo=True`` tells SQLAlchemy to log generated SQL and bound parameters for every request.

Kind	Name	Line
function	create_database_if_not_exists	32
function	seed_reference_data	76
function	get_db	147
function	init_db	155

backend/app/main.py

Line count: 59

Purpose: Application entrypoint. Builds the FastAPI object, configures CORS, mounts the versioned router, and performs schema/bootstrap work during startup.

- The FastAPI instance is created once at import time. This is the object Uvicorn looks for when you run ``uvicorn app.main:app``.
- CORS is narrowed to the configured frontend URL. That prevents the browser from rejecting cross-origin API calls coming from the Next.js frontend.
- The startup hook calls ``create_database_if_not_exists()``, imports the model package so SQLAlchemy metadata is fully populated, runs ``Base.metadata.create_all(bind=engine)``, and then seeds lookup rows.
- Because ``engine`` is configured with ``echo=True``, SQLAlchemy logs the SQL it emits. That is why startup often prints metadata checks and ``CREATE TABLE`` / ``SELECT`` statements.

Kind	Name	Line
function	on_startup	28
function	read_root	40
function	health_check	52

backend/app/models/__init__.py

Line count: 512

Purpose: Core domain model package. Defines users, projects, issues, statuses, priorities, components, versions, labels, comments, worklogs, boards, sprints, links, permissions, notifications, history, and favorites.

- The file uses SQLAlchemy 2-style typed mappings (``Mapped[...]` and ``mapped_column(...)``) together with classic ``relationship(...)`` declarations.
- Association tables such as ``issue_labels``, ``role_permissions``, and ``issue_sprints`` model many-to-many relationships.
- Convenience ``@property`` methods like ``project_key``, ``priority_name``, ``label_names``, and ``status_name`` make downstream serialization more readable.
- The Issue model is the center of gravity: nearly every major feature relates back to it directly or indirectly.

Kind	Name	Line
class	User	40

Kind	Name	Line
class	Project	66
class	IssueType	88
class	IssuePriority	101
class	IssueStatus	114
class	Resolution	129
class	Component	140
class	Version	157
class	Label	174
class	Issue	185
function	project_key	239
function	issue_type_name	243
function	priority_name	247
function	assignee_username	251
function	component_name	255
function	version_name	259
function	label_names	263
function	reporter_username	267
function	reporter_display_name	271
function	time_spent_hours	275
function	status_name	279
function	resolution_name	283
function	component_description	287
function	version_released	291
class	IssueComment	295
function	username	310
function	display_name	314
class	IssueAttachment	318
function	uploaded_by	335
class	Worklog	339
function	username	357
function	display_name	361
class	Board	365
function	project_key	383
class	BoardColumn	387
function	status_name	403
function	status_color	407
class	Sprint	411
class	IssueLink	430
class	ProjectRole	443
class	Permission	458

Kind	Name	Line
class	Notification	469
class	IssueHistory	486
class	Favorite	502

backend/app/models/agents.py

Line count: 42

Purpose: Persistence models for agent memory and agent action approvals.

- Agent memories store conversational context and workflow summaries for later inspection.
- Agent actions capture deferred tasks that need a human decision, such as approving an assignment recommendation.

Kind	Name	Line
class	AgentActionStatus	10
class	AgentMemory	17
class	AgentAction	29

backend/app/models/assignment.py

Line count: 42

Purpose: Data structures used by assignment intelligence.

- User skills describe each person's strengths.
- Assignment history lets the recommender reward users who have already worked in a project's context.
- Issue requirements store inferred skill needs extracted from issue text.

Kind	Name	Line
class	UserSkill	9
class	UserAssignmentHistory	20
class	IssueRequirement	34

backend/app/models/code_review.py

Line count: 54

Purpose: Persistence for PR linkage, code review findings, and bug reports created from those findings.

- This is the code-quality branch of the data model.
- A linked PR can generate multiple code review findings and multiple bug reports.

Kind	Name	Line
class	PullRequest	9
class	CodeReview	26
class	BugReport	41

backend/app/schemas/__init__.py

Line count: 405

Purpose: Pydantic schemas and enums used for validation and response shaping.

- Enums such as `IssueTypeEnum`, `PriorityEnum`, `StatusEnum`, and `BoardTypeEnum` keep the API contract tight.
- Create/update/response schemas are split so incoming and outgoing payloads can evolve independently.

- The issue schema family now includes Epic metadata and the `IssueEpicUpdate` payload used by the dedicated parent-Epic endpoint.

Kind	Name	Line
class	IssueTypeEnum	8
class	PriorityEnum	16
class	StatusEnum	24
class	BoardTypeEnum	32
class	UserBase	38
class	UserCreate	44
class	UserUpdate	51
class	UserResponse	57
class	Config	63
class	UserLogin	67
class	Token	72
class	TokenData	77
class	ProjectBase	82
class	ProjectCreate	88
class	ProjectUpdate	92
class	ProjectResponse	98
class	Config	106
class	IssueBase	111
class	IssueCreate	126
class	IssueUpdate	130
class	IssueResponse	146
class	Config	157
class	IssueDetailResponse	161
class	CommentBase	171
class	CommentCreate	175
class	CommentResponse	179
class	Config	188
class	CommentUpdate	192
class	AttachmentResponse	197
class	Config	205
class	WorklogBase	210
class	WorklogCreate	216
class	WorklogUpdate	220
class	WorklogResponse	226
class	Config	236
class	BoardBase	241
class	BoardCreate	248
class	BoardUpdate	252

Kind	Name	Line
class	BoardResponse	258
class	Config	264
class	BoardColumnBase	268
class	BoardColumnCreate	276
class	BoardColumnResponse	280
class	Config	286
class	SprintBase	291
class	SprintCreate	299
class	SprintUpdate	303
class	SprintResponse	312
class	Config	319
class	LabelBase	324
class	LabelCreate	329
class	LabelUpdate	333
class	LabelResponse	338
class	Config	341
class	IssueLinkBase	346
class	IssueLinkCreate	351
class	IssueLinkResponse	355
class	Config	362
class	IssueEpicUpdate	366
class	MessageResponse	371
class	PaginatedResponse	375
class	PRCreate	382
class	AgentStatusResponse	388
class	AgentActionResponse	395
class	AgentWorkflowResponse	402

backend/app/services/agent_service.py

Line count: 778

Purpose: Main automation brain. Contains deterministic assignment logic, sprint and PR agents, executive summary synthesis, pending action gating, action execution, and prompt-based object creation.

- The assignment engine infers skill requirements from issue text, stores those requirements, looks at user skills and assignment history, and then produces ranked candidates.
- The workload cap ('MAX_ISSUES_PER_USER') is enforced before ranking so overloaded users are heavily penalized.
- The 'agentic workflow' is not a LangGraph graph object today; it is an internal orchestration function that calls the relevant agent functions in sequence and then optionally asks Ollama for a narrative executive summary.
- Pending actions are persisted as database rows. Approval is decoupled from recommendation generation, which is why agent actions can survive request boundaries.
- Prompt automation is a second branch of the automation system. It converts natural language into a plan, creates the requested project/board/issues, links issue chains, assigns owners, creates notifications, and optionally sends email.

- The module blends deterministic rules with optional AI rather than depending fully on an LLM. That hybrid design explains many of the conditional capability checks throughout the file.

Kind	Name	Line
class	AgentState	39
function	langgraph_available	43
function	infer_issue_requirements	52
function	ensure_issue_requirements	70
function	get_user_workload	86
function	_priority_rank	98
function	_score_user_for_issue	105
function	recommend_issue_assignee	141
function	assignment_agent	190
function	pr_health_agent	217
function	sprint_agent	252
async function	executive_agent	282
function	pending_action_gate	318
function	execute_actions	354
function	_save_memory	397
async function	run_agentic_workflow	410
async function	run_conversational_agent	445
function	_default_board_columns	478
function	_create_project_from_plan	498
function	_create_board_from_plan	526
function	_create_issue_from_plan	547
function	_resolve_user_from_hint	589
function	_assign_issue_and_notify	627
function	_create_issues_from_plan	700
function	_link_issue_chain	713
async function	run_prompt_automation	732

backend/app/services/code_analyzer.py

Line count: 148

Purpose: Static-rule code analysis for linked pull requests.

- Fetches PR file contents, scans them against rule dictionaries, persists findings as `CodeReview` rows, and turns serious findings into bug reports.
- The rules are pattern-based rather than AST-based, so they are easy to extend but can generate false positives.
- The health score starts at 100 and is penalized according to finding severity.

Kind	Name	Line
function	analyze_code_from_github	47
function	create_bugs_from_findings	74
function	_analyze_file	93

Kind	Name	Line
function	_calculate_health_score	110
function	_fetch_pr_files	118

backend/app/services/email_service.py

Line count: 43

Purpose: SMTP integration for assignment notifications.

- It is intentionally thin: configuration check, generic `send\_email`, and a domain-specific `send\_issue\_assignment\_email` wrapper.
- The return values are dictionaries instead of exceptions so higher-level services can report partial success cleanly.

Kind	Name	Line
function	email_is_configured	7
function	send_email	11
function	send_issue_assignment_email	32

backend/app/services/langchain_service.py

Line count: 228

Purpose: Natural-language prompt planner for automation.

- If LangChain and a usable Ollama model are available, the planner asks the model to return strict JSON for project, board, issues, and email actions.
- If AI planning is not available or JSON parsing fails, a regex-driven fallback parser extracts project names, keys, issue types, priorities, assignee hints, auto-assign requests, and email intent.
- The normalizer stage ensures downstream automation code always sees the same shape even if the original plan came from different paths.

Kind	Name	Line
function	langchain_available	13
function	build_automation_plan	17
function	_build_plan_with_langchain	25
function	_fallback_plan	49
function	_extract_issue_items	119
function	_normalize_plan	178
function	_derive_project_key	220

backend/app/services/ollama_service.py

Line count: 133

Purpose: Local-LLM integration layer.

- Searches common install paths for the Ollama executable, then uses `ollama list` to discover installed models.
- `choose\_best\_model()` prefers a curated list but falls back to the first installed model.
- Both non-streaming and streaming chat helpers are provided. If Ollama or `httpx` is unavailable, the service degrades gracefully instead of crashing the app.

Kind	Name	Line
function	_locate_ollama	17

Kind	Name	Line
function	get_installed_models	31
function	choose_best_model	48
function	choose_best_embedding_model	58
async function	generate_response	63
async function	generate_response_stream	97

backend/app/services/sprint_service.py

Line count: 69

Purpose: Small service module for sprint lifecycle automation.

- Creates future sprints, activates future sprints whose start date has arrived, and closes active sprints whose end date has passed.
- This logic is intentionally deterministic and is reused by the automation workflow.

Kind	Name	Line
function	create_sprint_for_board	10
function	activate_current_sprint	37
function	close_completed_sprint	54